

实验报告：制造、解决并解释一次合并冲突

项目	内容
课程名称	系统开发工具基础
实验主题	Version Control and Git
题目编号	第3题
学号	25020007191
姓名	周御基
实验日期	2026年8月31日

一、实验目的

1. 掌握从零初始化 Git 仓库并完成初始提交的流程
2. 理解分支（branch）与合并（merge）的基本机制
3. 掌握制造一次合并冲突的方法，理解冲突产生的原因
4. 学会手动解决冲突并正确提交

二、实验环境

- 操作系统：Ubuntu Linux（远程服务器 192.168.79.130）
- 命令行工具：Git
- 远程连接：SSH（paramiko）

三、实验步骤与代码

步骤1：初始化 Git 仓库与初始提交

```
mkdir -p ~/q03 && git init -b main
git config user.email "zhou@example.com"
git config user.name "zhou"
printf 'mode=normal\n' > config.txt
git add config.txt && git commit -m "init: mode=normal"
```

输出结果：

```
Initialized empty Git repository in /home/joe/q03/.git/
( config.txt 内容 : mode=normal )
[main 5bfb9d1] init: mode=normal
```

步骤2：创建 feature-a，修改为 mode=safe 并提交

```
git checkout -b feature-a
printf 'mode=safe\n' > config.txt
git add config.txt && git commit -m "feature-a: mode=safe"
```

输出结果：

```
[feature-a 88ae272] feature-a: mode=safe
```

步骤3：从初始 main 创建 feature-b，修改为 mode=fast 并提交

```
git checkout main
git checkout -b feature-b main
printf 'mode=fast\n' > config.txt
git add config.txt && git commit -m "feature-b: mode=fast"
```

输出结果：

```
[feature-b a81dcb7] feature-b: mode=fast
```

步骤4: 回到 main, 先合并 feature-a

```
git checkout main
git merge --no-ff -m "merge feature-a" feature-a
cat config.txt
```

输出结果:

```
Merge made by the 'ort' strategy.
 config.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
( config.txt 现在是 mode=safe )
```

步骤5: 再合并 feature-b, 制造冲突

```
git merge feature-b
```

输出结果:

```
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
```

查看冲突文件内容及状态:

```
cat config.txt
git status
```

输出结果:

```
<<<<<<< HEAD
mode=safe
=====
mode=fast
>>>>>>> feature-b

On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file> ..." to mark resolution)
    both modified:   config.txt
```

冲突原因分析： feature-a 和 feature-b 都基于初始 `mode=normal` 的提交，分别把 config.txt 的第一行改成了 `mode=safe` 和 `mode=fast`。当 main 先合入 feature-a (变为 `mode=safe`) 后，再尝试合入 feature-b 时，两条分支都对同一行做了不同修改，Git 无法自动判断保留哪个，于是产生冲突，并在文件中标记 `<<<<<<<`、`=====`、`>>>>>>>` 三个冲突标记。

步骤6：解决冲突，保留 `mode=safe`，并另加 `note=reviewed`

```
printf 'mode=safe\nnote=reviewed\n' > config.txt
git add config.txt
git commit -m "resolve conflict: keep mode=safe, add note=reviewed"
```

输出结果：

```
[main 152d900] resolve conflict: keep mode=safe, add note=reviewed
```

解决方式：删除冲突标记，按题目要求保留 `mode=safe`（即 feature-a 的值），并新增一行 `note=reviewed`。

步骤7：确认工作区干净并查看历史

```
git status
git log --all --graph --decorate --oneline
```

输出结果：

```
On branch main
nothing to commit, working tree clean

* 152d900 (HEAD → main) resolve conflict: keep mode=safe, add note=reviewed
|
| * a81dcb7 (feature-b) feature-b: mode=fast
* | 660bf02 merge feature-a
|
|
| |
| |
| |
| * 88ae272 (feature-a) feature-a: mode=safe
|/
* 5bfb9d1 init: mode=normal
```

四、实验结果

4.1 结果展示

冲突解决后 config.txt 内容：

```
mode=safe
note=reviewed
```

工作区干净（nothing to commit, working tree clean），`git log --all --graph` 清晰地展示了三条分支的合并历史与合并提交节点。

4.2 结果分析

1. 合并冲突本质上是两条分支对同一文件同一位置做了不同修改，Git 无法自动合并，需要人工裁决。
2. 冲突文件中的三个标记 `<<<<<< HEAD`、`=====`、`>>>>>> feature-b` 分别表示当前分支、分隔线、被合并分支的版本。
3. 解决冲突的正确流程：编辑文件去掉冲突标记并选择内容 → `git add` 标记为已解决 → `git commit` 完成合并。
4. 合并后的历史图中，feature-a 的合并是 `--no-ff` 产生的真实合并提交（660bf02），feature-b 的冲突合并提交（152d900）呈“分叉再汇合”结构。

五、实验总结

通过本次实验，我掌握了以下技能：

1. `git init` 初始化仓库并配置 user 信息、完成初始提交
 2. 分支创建（`git checkout -b`）与切换
 3. 合并冲突的制造与成因分析
 4. 手动解决冲突（编辑文件、`git add`、`git commit`）的完整流程
 5. 使用 `git log --all --graph` 查看分支合并历史图
-

实验中的脚本文件：

1. `Scripts/q03/q03_git_conflict.sh`